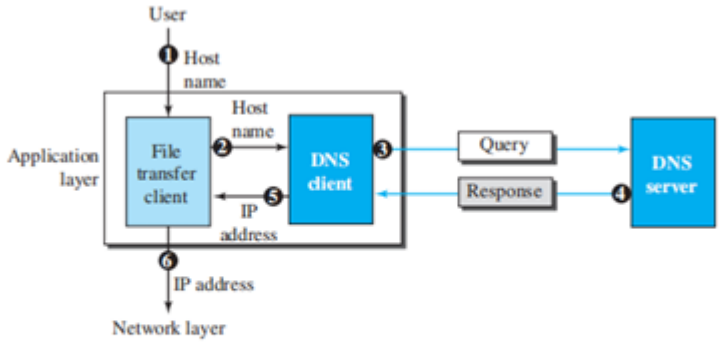
1. Explain DNS with architecture.

DOMAIN NAME SYSTEM (DNS)

The Domain Name System (DNS) was developed to simplify access to Internet resources by mapping human-friendly names to IP addresses, which are needed for network identification. Similar to how a telephone directory helps map names to numbers, DNS serves as a directory for the Internet, allowing users to remember domain names rather than numeric IP addresses. A central directory for the entire Internet would be impractical due to its vast scale and vulnerability to failures. Instead, DNS information is distributed across multiple servers worldwide. When a host requires name-to-IP mapping, it contacts the nearest DNS server with the necessary information. This distributed structure enhances reliability and efficiency.

Figure shows how TCP/IP uses a DNS client and a DNS server to map a name to an address. A user wants to use a file transfer client to access the corresponding file transfer server running on a remote host. The user knows only the file transfer server name, such as *afilesource.com*. The TCP/IP suite needs the IP address of the file transfer server to make the connection.



The following six steps map the host name to an IP address:

1. The user passes the host name to the file transfer client.
2. The file transfer client passes the host name to the DNS client.
3. Each computer, after being booted, knows the address of one DNS server. The DNS client sends a message to a DNS server with a query that gives the file transfer server name using the known IP address of the DNS server.
4. The DNS server responds with the IP address of the desired file transfer server.
5. The DNS server passes the IP address to the file transfer client.
6. The file transfer client now uses the received IP address to access the file transfer server.

2. Explain Persistent and Non-Persistent connection in HTTP.

Non-Persistent and Persistent Connections

If Separate TCP connection is used for each request and response, then the connection is said to be **non-persistent**.

If same TCP connection is used for series of related request and response, then the connection is said to be **persistent.**

Nonpersistent Connections

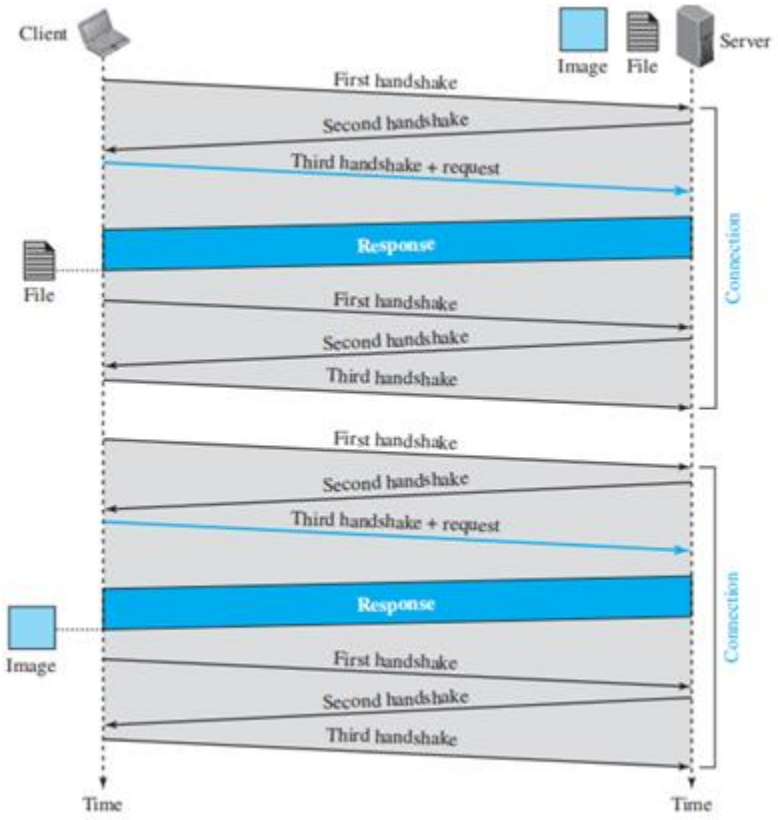
In a **nonpersistent connection**, one TCP connection is made for each request/response. The following lists the steps in this strategy:

1. The client opens a TCP connection and sends a request.

2. The server sends the response and closes the connection.

3. The client reads the data until it encounters an end-of-file marker; it then closes the connection.

In this strategy, if a file contains links to N different pictures in different files (all located on the same server), the connection must be opened and closed $N + 1$ times. The nonpersistent strategy imposes high overhead on the server because the server needs $N + 1$ different buffers each time a connection is opened.



- Round-trip time (RTT) is the time it takes for a small packet to travel from client to server and then back to the client.
- The RTT includes packet-propagation delays, packet queuing delays in intermediate routers and switches, and packet-processing delays.
- When a user clicks on a hyperlink, the browser initiates a TCP connection between the browser and the Web server; this involves a "three-way handshake"—the client sends a small TCP segment to the server, the server acknowledges and responds with a small TCP segment, and, finally, the client acknowledges back to the server.

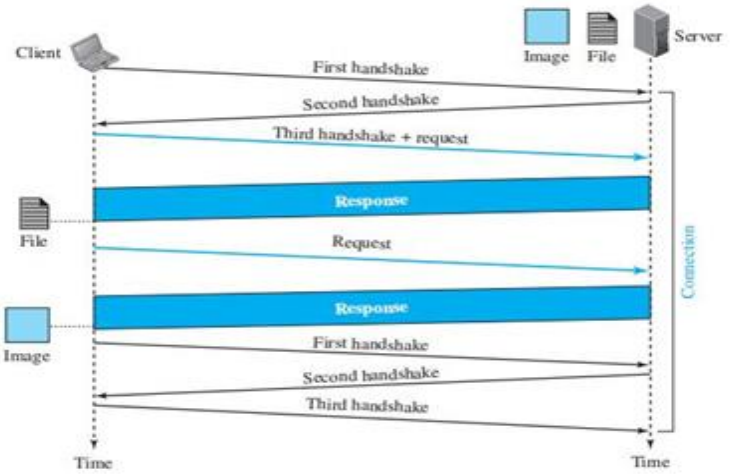
- The first two parts of the three-way handshake take one RTT.
- After completing the first two parts of the handshake, the client sends the HTTP request message combined with the third part of the three-way handshake (the acknowledgment) into the TCP connection.
- Once the request message arrives at the server, the server sends the HTML file into the TCP connection. This HTTP request/response eats up another RTT. Thus, roughly, the total response time is two RTTs plus the transmission time at the server of the HTML file.

Persistent Connections

HTTP version 1.1 specifies a persistent connection by default. Non-persistent connections have some shortcomings.

1. A brand-new connection must be established and maintained for each requested object. This can place a significant burden on the Web server, which may be serving requests from hundreds of different clients simultaneously.
2. Each object suffers a delivery delay of two RTTs— one RTT to establish the TCP connection and one RTT to request and receive an object.

With persistent connections, the server leaves the TCP connection open after sending a response. Subsequent requests and responses between the same client and server can be sent over the same connection. In particular, an entire Web page can be sent over a single persistent TCP connection. Moreover, multiple Web pages residing on the same server can be sent from the server to the same client over a single persistent TCP connection.



Only one connection establishment and connection termination is used, but the request for the image is sent separately

3. Explain Web Documents.

Web Documents

The documents in the WWW can be grouped into three broad categories: static, dynamic, and active

Static Documents

Static documents are fixed-content documents that are created and stored in a server. When a client accesses the document, a copy of the document is sent. The user can then use a browser to see the document. Static documents are prepared using one of several languages: *HyperText Markup Language* (HTML), *Extensible Markup Language* (XML), *Extensible Style Language* (XSL), and *Extensible Hypertext Markup Language* (XHTML).

Dynamic Documents

A **dynamic document** is created by a web server whenever a browser requests the document. When a request arrives, the web server runs an application program or a script that creates the dynamic document. The server returns the result of the program or script as a response to the browser that requested the document. Because a fresh document is created for each request, the contents of a dynamic document may vary from one request to another. A very simple example of a dynamic document is the retrieval of the time and date from a server.

Active Documents

Active documents are files that require a program or script to run on the client's device to enable interactive or animated content on the user's screen. When a browser requests an active

document, the server sends a copy of the file to the client. The document is then run by the client's browser, allowing animations and interactions to occur locally on the user's device.

Active Documents are created using two methods

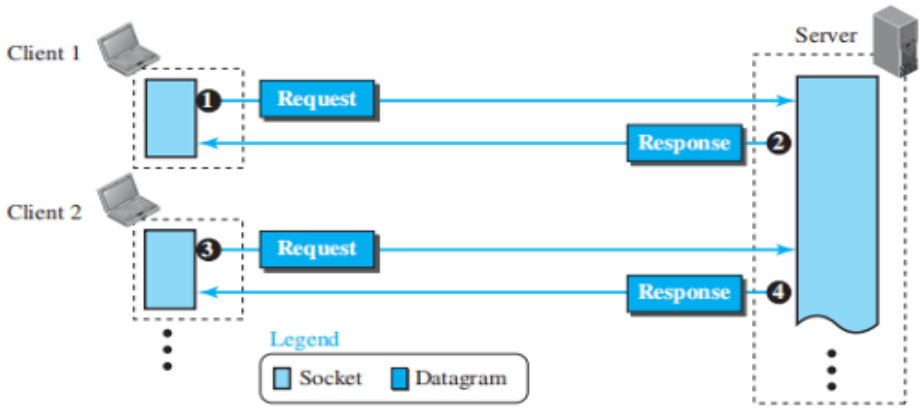
1. **Java Applets**: Written in Java and compiled into bytecode (binary format). Ready to run on the client's device.
2. **JavaScript**: Scripts that are downloaded from the server and executed directly by the client's browser.

These methods enable dynamic, interactive content directly on the client's device, making it possible for applications like animations and user interactions to work smoothly at the client side.

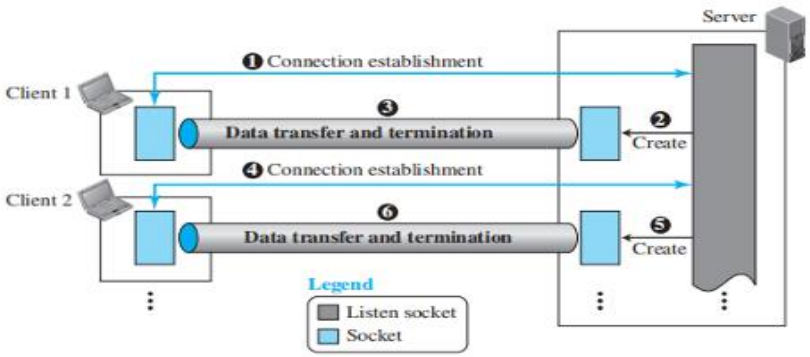
4. Bring out the difference Interactive communication using UDP and TCP.

Difference between Interactive Communication using UDP and TCP (5 Marks)

Point	UDP (User Datagram Protocol)	TCP (Transmission Control Protocol)
1. Connection Type	Connectionless – No connection is established before sending data.	Connection-oriented – A connection is established using a 3-way handshake.
2. Reliability	Unreliable – No guarantee of delivery, no acknowledgment, no retransmission.	Reliable – Ensures delivery through ACKs, retransmissions, error control.
3. Communication Speed	Faster, low-latency communication suitable for real-time interactions.	Slower due to connection setup, reliability features, and flow control.
4. Message Handling	Message-oriented; preserves message boundaries.	Byte-stream oriented; message boundaries are not preserved.
5. Use in Interactive Communication	Suitable for time-sensitive interactive services (e.g., VoIP, video conferencing, online gaming). Data may be lost but delay must be minimal.	Suitable for applications requiring accurate, complete data (e.g., remote login via SSH/Telnet, file transfer). Delay is acceptable but loss is not.



UDP



TCP

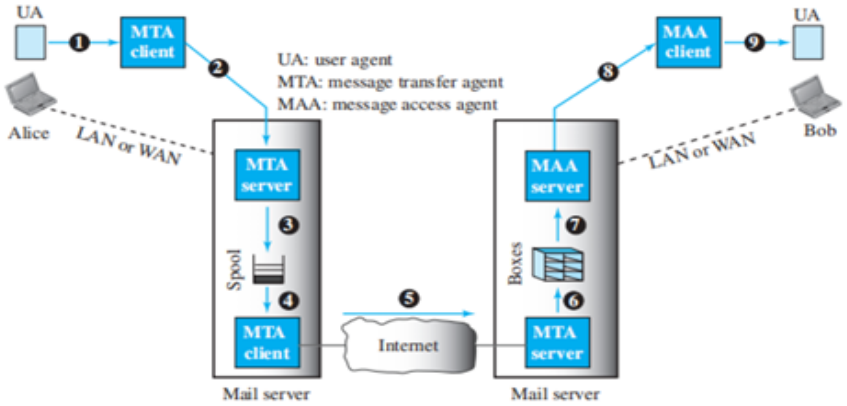
5. Explain the working model of Electronic Mail

ELECTRONIC MAIL

Electronic mail (or e-mail) allows users to exchange messages. In applications like HTTP or FTP, the server constantly awaits client requests to provide a response, whereas in email, messages are sent and stored until the recipient retrieves them. when Alice sends an email to Bob, a response is optional and, if given, is a separate transaction. Unlike typical client-server setups, Bob doesn't run a server to receive emails, as he might be offline. Instead, intermediate servers handle the client-server functions, allowing users to connect only when they wish to check or send messages.

Architecture

To explain the architecture of email, we present a typical scenario, as illustrated in Figure. An alternative scenario occurs when Alice or Bob is directly connected to their respective mail server, eliminating the need for a LAN or WAN connection; however, this variation does not affect the overall discussion.



In the common scenario, the sender and the receiver of the e-mail, Alice and Bob respectively, are connected via a LAN or a WAN to two mail servers. The administrator has created one mailbox for each user where the received messages are stored. A *mailbox* is part of a server hard drive, a special file with permission restrictions. Only the owner of the mailbox has access to it. The administrator has also created a queue (spool) to store messages waiting to be sent.

A simple e-mail from Alice to Bob takes nine different steps, as shown in the figure. Alice and Bob use three different *agents*: a **user agent (UA),** a **message transfer agent (MTA),** and a **message access agent (MAA).**

1. Alice composes an email using her User Agent (UA).
2. The UA sends the message to the Message Transfer Agent (MTA) client on Alice's mail server.
3. The MTA client forwards the message to the MTA server on the same mail server.
4. The email is placed in a spool (queue) on Alice's mail server, awaiting transfer.
5. The MTA client sends the email across the Internet to Bob's mail server.
6. The MTA server on Bob's mail server receives the email.
7. The email is stored in Bob's mailbox on his mail server.
8. Bob's Message Access Agent (MAA) server retrieves the message from the mailbox.
9. Bob uses his MAA client to access the message through his User Agent (UA).

6. How FTP is maintained in the Application layer for client and server

1. Control Connection (Port 21)

- This connection is established between the **client control process** and the **server control process.**
- It remains **open throughout the entire FTP session.**
- Used for sending **commands** from client to server (like USER, PASS, GET, PUT) and **responses** from server to client.
- Communication is simple, text-based, one line at a time.

2. Data Connection (Port 20)

- Used to **transfer actual files** (upload/download) or directory listings.
- This connection is opened **only when data needs to be transferred,** and closed afterward.
- Provides reliable file transfer using TCP.

3. FTP Client Structure

The FTP client has three parts:

1. **User Interface** – allows the user to input commands.
2. **Client Control Process** – sends commands to the server using the control connection.
3. **Client Data Transfer Process** – receives or sends files over the data connection.

4. FTP Server Structure

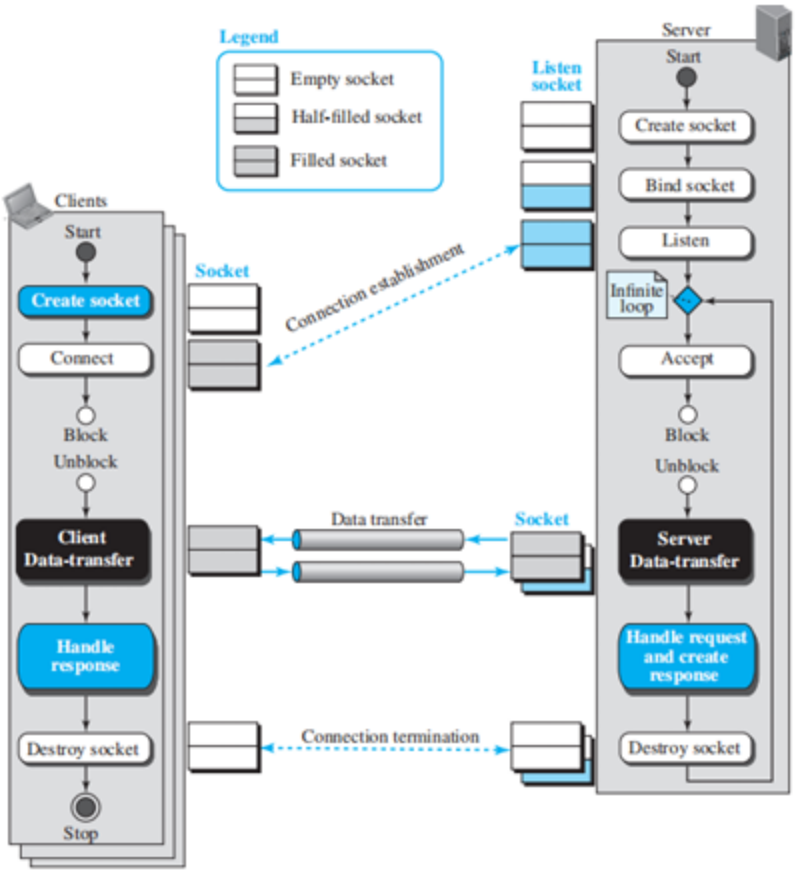
The server has two components:

1. **Server Control Process** – listens on port 21, receives commands, and sends status codes.
2. **Server Data Transfer Process** – handles the actual file transfer via port 20.

5. Working Flow (Step-by-Step)

1. Client opens a TCP connection to server at **port 21** → Control connection established.
2. User logs in using FTP commands.
3. For every file transfer, the server opens a **data connection** (port 20).
4. File/data is transferred through the data connection.
5. Data connection closes after transfer; control connection remains active.
6. After all operations, client sends QUIT command and control connection closes.

7. Draw flow diagram for the client-server application using TCP. Also, write code for the client-&server-sides of the application.



Server Process

- The TCP server creates a listening socket and binds it to a network address and port, used only for listening to incoming client connections.
- The server calls a "listen" function, allowing the operating system to accept connections and add them to a waiting list.
- The server enters a loop, handling each client one at a time.
- In each loop iteration, the server uses the "accept" function to pick a client from the waiting list; if no clients are waiting, the server blocks until a client arrives.
- Upon accepting a client, a new socket is created specifically for data exchange with that client.
- The server sets this new socket with the client's address for direct communication.
- The server and client can then exchange data using this client-specific socket.

Client Process

The client flow diagram is almost similar to the UDP version except that the *client data-transfer* box needs to be defined for each specific case.

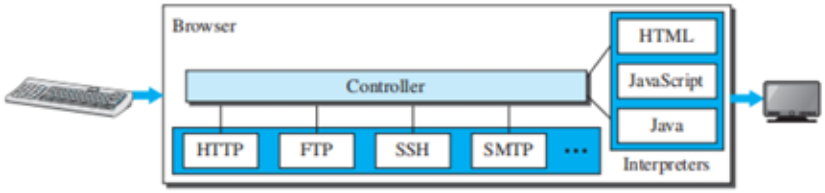
8. With a neat Architecture diagram explain World Wide Web

Architecture

The WWW today is a distributed client-server service, in which a client using a browser can access a service using a server. However, the service provided is distributed over many locations called **sites**. Each site holds one or more **web pages**. A web page can be **simple or composite**. A simple web page has no links to other web pages; a composite web page has one or more links to other web pages. Each web page is a file with a name and address.

Web Client (Browser)

A variety of vendors offer commercial **browsers** that interpret and display a web page, and all of them use nearly the same architecture. Each browser usually consists of three parts: a controller, client protocols, and interpreters.



- The controller receives input from the keyboard or the mouse and uses the client programs to access the document.
- After the document has been accessed, the controller uses one of the interpreters to display the document on the screen.
- The client protocol can be one of the protocols described later, such as HTTP or FTP.
- The interpreter can be HTML, Java, or JavaScript, depending on the type of document.
- Some commercial browsers include Internet Explorer, Netscape Navigator, and Firefox.

Web Server

- The web page is stored at the server. Each time a request arrives, the corresponding document is sent to the client.
- To improve efficiency, servers normally store requested files in a cache in memory; memory is faster to access than a disk.
- A server can also become more efficient through multithreading or multiprocessing. In this case, a server can answer more than one request at a time.
- Some popular web servers include Apache and Microsoft Internet Information Server.

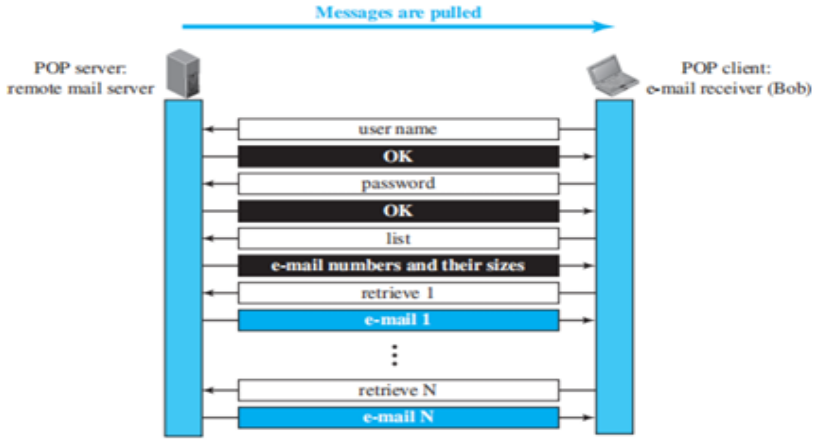
9. Mention the difference between Static Document and Dynamic Document in Web

Static Document vs Dynamic Document	
Static Document	**Dynamic Document**
Content is **fixed** and does **not change** unless manually updated by the developer.	Content is **generated at runtime** based on user request or server-side processing.
Stored **as-is** on the web server.	Created **fresh for each request** using application programs or scripts.
Examples: HTML pages, images, CSS files.	Examples: pages showing current date/time, search results, user profiles.
Faster to load since no server-side processing is required.	Slower than static pages because server processes scripts/programs before sending output.
Suitable for simple websites with less interaction.	Suitable for interactive websites like e-commerce, social media, and portals.

10. With a neat sketch explain POP3, how it impact on IMAP4 and MIME

POP3

Post Office Protocol, version 3 (POP3) is simple but limited in functionality. The client POP3 software is installed on the recipient computer; the server POP3 software is installed on the mail server. Mail access starts with the client when the user needs to download its e-mail from the mailbox on the mail server. The client opens a connection to the server on TCP port 110. It then sends its user name and password to access the mailbox. The user can then list and retrieve the mail messages, one by one.



POP3 has two modes: the ***delete*** mode and the ***keep*** mode. In the delete mode, the mail is deleted from the mailbox after each retrieval. In the keep mode, the mail remains in the mailbox after retrieval.

IMAP4

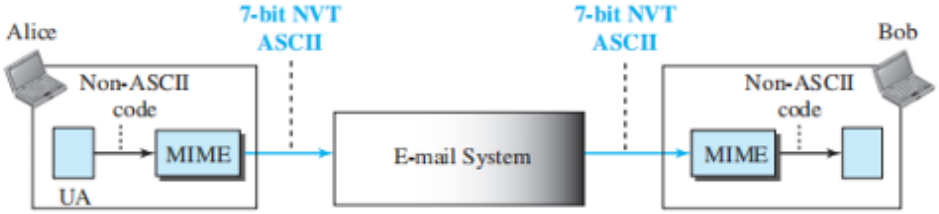
Another mail access protocol is **Internet Mail Access Protocol, version 4 (IMAP4)**. IMAP4 is similar to POP3, but it has more features; IMAP4 is more powerful and more complex.

POP3 is deficient in several ways. It does not allow the user to organize her mail on the server; the user cannot have different folders on the server. In addition, POP3 does not allow the user to partially check the contents of the mail before downloading. IMAP4 provides the following extra functions:

- A user can check the e-mail header prior to downloading.
- A user can search the contents of the e-mail for a specific string of characters prior
- to downloading.
- A user can partially download e-mail. This is especially useful if bandwidth is limited and the e-mail contains multimedia with high bandwidth requirements.
- A user can create, delete, or rename mailboxes on the mail server.
- A user can create a hierarchy of mailboxes in a folder for e-mail storage.

MIME

Email has a simple structure, but it is limited to sending messages in 7-bit ASCII format, which restricts its use for non-English languages and prevents sending binary files, video, or audio data.

Multipurpose Internet Mail Extensions (MIME) is a supplementary protocol that allows non-ASCII data to be sent through e-mail. MIME transforms non-ASCII data at the sender site to NVT ASCII data and delivers it to the client MTA to be sent through the Internet. The message at the receiving site is transformed back to the original data.

MIME is a set of software functions that transforms non-ASCII data to ASCII data and vice versa.